Simulador de Inserção de Jogadores

Guia educacional — arquitetura e Python passo a passo

Tópicos abordados
• Separação de responsabilidades (Model / View / Controller informal)
• Validação de dados com Pydantic v2
• Servidor HTTP em Flask (localhost)
• Persistência incremental em JSON com timestamp único
• Comunicação frontend ↔ backend via **fetch** + JSON
• Como estender o código sem quebrar o original

Nível: iniciante-intermediário | **Stack:** Python 3.13 · Flask · Pydantic v2

Autoria: Professora Paula Peclat

© 2026 — Todos os direitos reservados.

1. Visão geral do projeto

O simulador é uma pequena aplicação web que recebe uma lista de jogadores digitada no navegador, processa cada registro pela mesma pipeline do **completo.py** original (sanitização + validação + cálculo do índice de combate) e grava um **novo arquivo JSON** para cada envio.

A ideia arquitetural é dividir a aplicação em três responsabilidades bem separadas, cada uma em um arquivo próprio:

Camada	Arquivo	Responsabilidade
Domínio (regras)	**completo.py**	Validar entrada, aplicar regras de negócio e gravar o JSON. Não sabe que existe HTTP.
Transporte (HTTP)	**app.py**	Servidor Flask. Recebe requisições, chama funções do domínio, devolve resposta JSON.
Interface (UI)	**templates/index.html**	Formulário no navegador. Coleta os dados e faz *fetch* para o backend.

■ Essa separação chama-se **separation of concerns**. Cada arquivo tem **uma** razão para mudar. Se amanhã trocarmos Flask por FastAPI, **completo.py** não muda uma linha.

Estrutura de pastas

```
simulador/
■■■ completo.py          # pipeline de domínio (original + adições marcadas)
■■■ app.py               # servidor Flask em http://localhost:5000
■■■ requirements.txt     # dependências (flask, pydantic)
■■■ templates/
■   ■■■ index.html       # formulário multi-linhas para inserir jogadores
■■■ dados/               # criado no 1º envio; um JSON por inserção
```

2. completo.py — o coração do domínio

Este arquivo contém as regras de negócio. Ele foi mantido exatamente igual ao original que você já tinha, e as adições necessárias para o simulador foram inseridas entre marcadores explícitos:

```
# === INICIO NOVO ===
...
# === FIM NOVO ===
```

■ Marcar código adicionado com comentários delimitadores é uma técnica simples e útil quando você **não pode ou não deve reescrever** o código base — mostra ao leitor exatamente o que é seu.

2.1 Modelo de dados com Pydantic

A classe **Jogador** define a forma esperada de cada registro. Pydantic faz duas coisas importantes de uma vez:

- **Validação de tipo**: se você passar *"cinco"* em vez de **5**, ele levanta erro automaticamente.
- **Validação de regra**: o **@field_validator** garante que Mortes, Abates e Assistências não sejam negativos.

```
class Jogador(BaseModel):
    Nickname: str
    Mortes: int
    Abates: int
    Assistências: int

    @field_validator('Mortes', 'Abates', 'Assistências')
    @classmethod
    def validar_valores(cls, value: int, info) -> int:
        if value < 0:
            raise ValueError('...')
        return value
```

2.2 Sanitização antes da validação

Antes de tentar instanciar o modelo, **sanitizar_registro** corrige valores negativos para 0 e imprime um aviso. Isso é uma escolha de arquitetura: **tolerar** a entrada suja em vez de rejeitá-la.

```
def sanitizar_registro(registro: dict) -> dict:
    for campo in ['Mortes', 'Abates', 'Assistências']:
        if campo in registro and registro[campo] < 0:
            print(f"[AVISO] ... corrigido para 0.")
            registro[campo] = 0
    return registro
```

■ Sanitizar **antes** de validar é um padrão comum: você deixa a camada de validação estrita (só recebe dado limpo) e concentra o "perdão" numa função dedicada.

2.3 Regra de negócio: índice de combate

Função pura — dados os mesmos números, sempre devolve o mesmo resultado. Sem I/O, sem estado global. Isso é o que torna fácil **testar** essa função (basta chamar e comparar).

```
def indice_combate(Mortes, Abates, Assistências):
    divisor = 1 if Mortes == 0 else Mortes
    return (Abates + Assistências) / divisor
```

2.4 O bloco NOVO — processar + salvar

Duas funções curtas foram acrescentadas. Ambas usam apenas o que já existia no arquivo original — nada é reimplementado.

```
def _processar(r: dict) -> dict:
    r = sanitizar_registro(dict(r))
    try:
        j = Jogador(**r)
        return {
            'Nickname': j.Nickname, 'Mortes': j.Mortes,
            'Abates': j.Abates, 'Assistências': j.Assistências,
            'IndiceCombate': round(indice_combate(
                j.Mortes, j.Abates, j.Assistências), 2),
        }
    except (ValidationError, Exception) as e:
        return {**r, 'Erro': str(e)}

def processar_registros(registros: list[dict]) -> list[dict]:
    return [_processar(r) for r in registros]
```

Repare em três decisões:

1. **_processar** começa com underscore — convenção em Python para dizer "função interna, não é para uso externo".
2. Uma **list comprehension** substitui o laço explícito — mesma coisa, mais compacto.
3. Em caso de erro, o dicionário original é devolvido com uma chave extra **Erro**. Não perdemos a entrada.

```
def salvar_novo_json(resultados, pasta_destino='dados') -> str:
    pasta = Path(pasta_destino); pasta.mkdir(parents=True, exist_ok=True)
    caminho = pasta / f'insercao_{datetime.now():%Y%m%d_%H%M%S_%f}.json'
    caminho.write_text(
        json.dumps(resultados, ensure_ascii=False, indent=2),
        encoding='utf-8')
    return str(caminho)
```

■ O formato do timestamp inclui **%f** (microssegundos). Isso garante que **dois envios no mesmo segundo** gerem arquivos distintos — nada é sobrescrito.

3. app.py — o servidor Flask

Flask é um *micro-framework*: dá o mínimo (roteamento, request/response, templates) e deixa o resto por sua conta. Perfeito para uma aplicação pequena como esta.

3.1 Codificação e imports

```
import sys
if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8')

from pathlib import Path
from flask import Flask, request, render_template, jsonify, send_from_directory
from completo import processar_registros, salvar_novo_json
```

■ Ajustamos o stdout para UTF-8 sem **fechar** o stream original — o Flask/Click precisa dele intacto. Detalhe pequeno que evita um erro chato no Windows.

3.2 As rotas

Método	URL	O que faz
GET	/	Serve o formulário HTML.
POST	/inserir	Recebe a lista, processa, grava JSON, retorna resumo.
GET	/dados	Lista todos os JSONs já gerados.
GET	/dados/<nome>	Baixa o conteúdo de um JSON específico.

3.3 A rota /inserir por dentro

Esta é a rota principal. Ela aceita **duas formas** de entrada: JSON (quando chamada pelo *fetch* do navegador) ou um formulário URL-encoded (quando alguém envia o <form> sem JavaScript).

```
@app.route('/inserir', methods=['POST'])
def inserir():
    registros: list[dict] = []

    if request.is_json:
        payload = request.get_json(silent=True) or []
        registros = [payload] if isinstance(payload, dict) else payload
    else:
        nicks = request.form.getlist('Nickname')
        # ... monta cada dicionário a partir de campos repetidos

    if not registros:
        return jsonify({'erro': 'Nenhum registro válido enviado.'}), 400

    resultados = processar_registros(registros)
    caminho    = salvar_novo_json(resultados, pasta_destino=str(PASTA_DADOS))
    return jsonify({
```

```
        'arquivo_gerado': Path(caminho).name,
        'total': len(resultados),
        'resultados': resultados,
    })
```

Três observações valiosas para o aluno:

- A rota **não** sabe validar jogador — ela delega para **completo.py**. É uma "casca fina".
- O código de status **400** é devolvido quando o cliente envia lixo. Sempre distingua erro do cliente (4xx) de erro seu (5xx).
- A resposta é sempre JSON, mesmo quando dá erro — assim o frontend não precisa adivinhar o formato.

4. templates/index.html — o formulário

O HTML tem duas partes: a estrutura visual (tabela editável) e um pequeno trecho de JavaScript que empacota os dados e envia para o backend.

4.1 A tabela editável

Cada linha da tabela vira um jogador. Os botões **+ Adicionar** e **×** permitem crescer ou encolher a lista sem recarregar a página.

4.2 O envio assíncrono com fetch

```
const jogadores = [];
document.querySelectorAll('tbody tr').forEach(linha => {
    const inputs = linha.querySelectorAll('input');
    jogadores.push({
        Nickname: inputs[0].value.trim(),
        Mortes:   parseInt(inputs[1].value) || 0,
        Abates:   parseInt(inputs[2].value) || 0,
        'Assistências': parseInt(inputs[3].value) || 0,
    });
});

const res  = await fetch('/inserir', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(jogadores),
});
const data = await res.json();
```

■ **await** só funciona dentro de uma função **async**. Ele pausa a execução até o servidor responder, sem congelar a página.

Ao receber a resposta, o script monta uma tabela-resumo (Nickname, IndiceCombate, badge OK/erro) e mostra o JSON completo em um <details> recolhível.

5. O fluxo de uma inserção do começo ao fim

Etapa	Onde acontece	O que rola
1	Navegador	Usuário preenche a tabela e clica em **Enviar**.
2	index.html (JS)	O script varre as linhas, monta uma lista de dicionários e chama **fetch('/inserir')**.
3	Flask / app.py	**@app.route('/inserir')** recebe. Detecta que é JSON, pega o payload.
4	completo.py	**processar_registros** é chamada; para cada jogador aplica sanitização, valida com Pydantic e calcula o índice.
5	completo.py	**salvar_novo_json** grava **dados/insercao_<timestamp>.json**.
6	Flask / app.py	A rota devolve **jsonify({...})** com o nome do arquivo e a lista processada.
7	Navegador	O JS renderiza a tabela-resumo e o JSON dentro do <details>.

■ Observe que os dados **mudam de forma** em cada camada: HTML/inputs → objeto JS → JSON de rede → dict Python → objeto Pydantic → dict de saída → JSON no disco. Cada troca é uma oportunidade de validar e transformar.

6. Conceitos de arquitetura para levar dessa aula

Separação de responsabilidades

Se o Flask sumisse amanhã, você poderia importar **completo.py** em um script de linha de comando, em um notebook Jupyter ou em uma tarefa agendada — **sem mudar nada** no arquivo. Isso só é possível porque as funções de domínio não conhecem HTTP.

Funções puras vs funções com efeito colateral

- **indice_combate**: função pura. Fácil de testar, fácil de reaproveitar, previsível.
- **salvar_novo_json**: função com efeito colateral (mexe no disco). Deve ser isolada e nomeada de forma que o efeito seja óbvio.

Idempotência e histórico

Gravar um novo arquivo a cada inserção, em vez de sobrescrever um único arquivo, transforma a pasta **dados/** em um **log**. Você pode voltar no tempo, comparar envios, auditar. É um padrão simples chamado **event sourcing** na sua forma mais básica.

Contrato entre camadas

A rota **/inserir** promete: *eu recebo uma lista de dicionários e devolvo uma resposta JSON com resultados*. Se você respeitar esse contrato, pode trocar a UI (HTML, mobile, CLI) sem tocar no backend, e vice-versa.

Estender sem quebrar

Manter o código original intacto e delimitar as adições com **# === INICIO NOVO ===** / **# === FIM NOVO ===** é uma disciplina de humildade: você respeita o autor anterior e deixa claro para o próximo leitor onde termina o legado e onde começa o novo.

7. Como rodar o simulador

```
cd simulador
pip install -r requirements.txt
python app.py
# abrir http://localhost:5000
```

Após um envio bem-sucedido, verifique **simulador/dados/** — você verá um arquivo **insercao_<timestamp>.json** por clique. O conteúdo tem exatamente a mesma forma do **ficha_geral_player.json** original, com uma chave extra **IndiceCombate**.

8. Exercícios sugeridos

1. Adicione um novo campo **ArmaFavorita** ao modelo Pydantic e ao formulário. Quais arquivos precisam mudar? (dica: pelo menos três)
2. Escreva um **teste unitário** para **indice_combate** cobrindo o caso de Mortes = 0. Use **pytest** ou **unittest**.
3. Crie uma nova rota **GET /ranking** que leia todos os arquivos em **dados/** e devolva os 10 maiores **IndiceCombate** agregados.
4. O arquivo **salvar_novo_json** usa timestamp com microssegundos. Simule um cenário em que dois envios chegam **exatamente no mesmo microssegundo**. Como você garantiria unicidade?
5. Refatore o <script> do **index.html** extraindo o código em um arquivo separado **static/simulador.js** — o que muda no HTML? Por que isso pode ser melhor?

■ Toda vez que um exercício mudar mais de uma camada, pergunte-se: *essas camadas estão fracamente acopladas ou eu misturei responsabilidades?*

9. Gabarito comentado

Confira sua solução **depois** de tentar. Ler a resposta antes de praticar é como assistir ao replay antes de jogar a partida — você não constrói o reflexo.

9.1 Campo ArmaFavorita

Três arquivos mudam: **completo.py** (modelo), **index.html** (coluna e input) e opcionalmente **app.py** (parsing do formulário).

Em **completo.py**, dentro da classe **Jogador**:

```
class Jogador(BaseModel):
    Nickname: str
    Mortes: int
    Abates: int
    Assistências: int
    ArmaFavorita: str | None = None    # opcional — não quebra dados antigos
```

Ajuste também o dicionário devolvido por _**processar** para incluir **'ArmaFavorita': j.ArmaFavorita**.

Em **index.html**, adicione <th> e <input name="ArmaFavorita"> na tabela, e inclua no objeto enviado pelo fetch:

```
jogadores.push({
    Nickname: inputs[0].value.trim(),
    Mortes:   parseInt(inputs[1].value) || 0,
    Abates:   parseInt(inputs[2].value) || 0,
    'Assistências': parseInt(inputs[3].value) || 0,
    ArmaFavorita: inputs[4].value.trim() || null,
});
```

■ Tornar o campo **opcional** (**str | None**) evita quebrar os JSONs antigos em **dados/** se você quiser lê-los depois. Compatibilidade retroativa é decisão de arquitetura.

9.2 Teste unitário de indice_combate

Crie **tests/test_completo.py**:

```
from completo import indice_combate

def test_mortes_zero_nao_divide_por_zero():
    # 20 abates + 5 assistências, sem morrer: divisor vira 1
    assert indice_combate(0, 20, 5) == 25.0

def test_calculo_normal():
    # (18 + 6) / 3 = 8.0
    assert indice_combate(3, 18, 6) == 8.0

def test_zeros_totais():
    assert indice_combate(0, 0, 0) == 0.0
```

Rode com **pytest -v tests/**. A vantagem: como **indice_combate** é uma **função pura**, o teste é trivial — não precisa mockar HTTP, banco nem disco.

9.3 Rota GET /ranking

Adicione em **app.py**:

```
@app.route('/ranking', methods=['GET'])
def ranking():
    todos = []
    for arq in PASTA_DADOS.glob('insercao_*.json'):
        dados = json.loads(arq.read_text(encoding='utf-8'))
        todos.extend(r for r in dados if 'IndiceCombate' in r)

    top10 = sorted(todos,
                   key=lambda r: r['IndiceCombate'],
                   reverse=True)[:10]
    return jsonify({'top': top10, 'total_analisado': len(todos)})
```

Não esqueça o **import json** no topo. O **if 'IndiceCombate' in r** descarta registros que falharam na validação (tinham chave **Erro**).

■ Se a pasta **dados/** crescer muito, ler todos os arquivos a cada request vira gargalo. Solução profissional: manter um índice em memória ou usar um banco (SQLite basta). Por enquanto, está ótimo.

9.4 Colisão de timestamp

Duas requisições no mesmo microssegundo são raras mas possíveis. Três estratégias, da mais simples à mais robusta:

1. **Sufixo aleatório**: acrescente **uuid.uuid4().hex[:6]** ao nome do arquivo. Rápido, sem coordenação.
2. **Contador atômico em memória**: use uma trava (**threading.Lock**) e incremente um contador global. Funciona só em um único processo.
3. **Criação atômica no disco**: use **open(caminho, 'x')** — que **falha** se o arquivo já existe — dentro de um laço que gera um novo nome. Robusto até entre processos.

Solução híbrida recomendada:

```
import uuid

def salvar_novo_json(resultados, pasta_destino='dados') -> str:
    pasta = Path(pasta_destino); pasta.mkdir(parents=True, exist_ok=True)
    ts = datetime.now().strftime('%Y%m%d_%H%M%S_%f')
    sufixo = uuid.uuid4().hex[:6]
    caminho = pasta / f'insercao_{ts}_{sufixo}.json'
    caminho.write_text(json.dumps(resultados, ensure_ascii=False, indent=2),
                       encoding='utf-8')
    return str(caminho)
```

9.5 JavaScript em arquivo separado

Estrutura resultante:

```
simulador/
■■■ static/
■   ■■■ simulador.js
■■■ templates/
    ■■■ index.html
```

No fim do **index.html**, substitua o <script> inline por:

```
<script src="{{ url_for('static', filename='simulador.js') }}" defer></script>
```

Ganhos:

- **Cache do navegador**: o JS é baixado uma vez e reaproveitado nas próximas visitas.
- **Content Security Policy**: mais fácil endurecer a segurança sem **unsafe-inline**.
- **Testabilidade**: dá para rodar linter e testes no JS isolado.
- **Separação de responsabilidades**: HTML descreve estrutura, JS descreve comportamento.

■ O Flask serve automaticamente qualquer arquivo dentro de **static/**. A helper **url_for('static', ...)** gera o caminho certo mesmo se o app rodar sob um prefixo diferente.

Fim — pratique construindo pequenos serviços como este. Cada projeto pequeno consolida um princípio que valerá para sistemas grandes.

Autoria e direitos

© 2026 **Professora Paula Peclat** — Todos os direitos reservados.

Este material é de autoria exclusiva de Paula Peclat e destina-se a uso educacional. Reprodução, distribuição ou adaptação sem autorização expressa da autora são vedadas.